

Apéndices técnicos de CREApp

Auditoría de ramas, contratos, riesgos y evidencia de validación

Código del Documento:	CREAPP-OPS-02
Versión:	3.0
Autor:	Juan Martín Morales
Repositorio:	https://github.com/juanm4morales/cre-app
Clasificación:	Documentación Técnica de Operaciones
Destinatarios:	Administradores de Sistemas, Infraestructura y DevOps
Fecha de Revisión:	30 de septiembre de 2026

Universidad Nacional de Cuyo — Sistema de Gestión Curricular y Equivalencias (CREApp)

Este documento reúne el detalle de respaldo de los manuales de despliegue y desarrollo.

Cobertura documental: servidores propios y Azure. Este documento no verifica infraestructura ni despliegues en ejecución.

Contenido

Propósito y alcance	1
1. Ramas y diferencias reales	1
2. Restricciones de la receta self-hosted	1
3. Seguridad y entorno	2
4. Migraciones con historial	2
5. Deuda técnica conocida	2
6. Contratos de serialización	3
7. Detalle del cliente frontend	3
8. Evidencia de las pruebas	4
9. Diagramas	4
10. Documentos relacionados	4

Propósito y alcance

Detalle de respaldo para los [manuales de despliegue](#) y [desarrollo](#). Aquí se registran las observaciones de auditoría, las diferencias entre ramas y los riesgos conocidos. Ninguna de estas afirmaciones describe el estado en vivo de Azure.

1. Ramas y diferencias reales

La intención de diseño es que **azure** sea la fuente de producto y que **azure-same-origin** sea una variante de despliegue derivada, con diferencias concentradas en routing y despliegue. En las instantáneas auditadas esa intención no se cumple: la variante también modifica el contrato CSRF y los workflows.

Snapshot observado	Rutas y estáticos	CSRF	Workflows de App Service
origin/azure	/api/static/, admin /api/sadmin-creapp- panel/, build con VITE_ API_URL=/api y VITE_ STATIC_BASE=/api/ static/	Devuelve csrfToken en JSON además de cookie; el cliente acepta ambos	deploy.yml y main_ cre-app-api.yml fil- tran main; deploy.yml también azure
origin/azure-same- origin	/static/, admin / sadmin-creapp-panel/ , base de Vite fija en / static/	Solo cookie; el cliente lee document.cookie	Ambos workflows filtran main y azure
chore/app-service- same-origin	/api/static/, admin /api/sadmin-creapp- panel/	Devuelve csrfToken en JSON	main_cre-app-api.yml filtra solo main

Consecuencias prácticas:

- La variante con rutas raíz no es un caso donde cambiar `VITE_STATIC_BASE` alcance: su base está fija en el código. Si se usa esa variante, el `alias` de estáticos y el proxy del panel de administración deben coincidir con ella, o el fallback de la SPA responderá HTML donde se espera el admin y puede producir un bucle de redirección.
- El token CSRF en JSON existe para el caso split-origin, donde la SPA no puede leer la cookie del backend. Con solo cookie, ese contrato no debe darse por compatible.
- Que dos workflows declaren la misma aplicación no prueba qué se despliega: las definiciones no determinan los workflows habilitados ni el artefacto publicado.
- El workflow de Static Web Apps tiene el `push` comentado y conserva los eventos de pull request. No es un despliegue por push.

Las ramas `main`, `dev`, `unidades`, `frontend-modernization` y `legacy/cross-site` están fusionadas y son de referencia. No son objetivos de compatibilidad ni rutas de despliegue.

2. Restricciones de la receta self-hosted

El ejemplo de Nginx y el build con `/api/static/` forman un contrato: `STATIC_URL=/api/static/`, panel en `/api/sadmin-creapp-panel/`, `VITE_API_URL=/api` y `VITE_STATIC_BASE=/api/static/`. Antes de adaptar la receta, revise las rutas reales de la versión que va a instalar.

El orden importa: el frontend se compila antes de `collectstatic` porque su salida forma parte de los estáticos de Django, y `current` se publica solo después de compilar, migrar y recolectar. Publicar antes de tiempo hace que Nginx sirva archivos nuevos mientras Gunicorn ejecuta el release anterior.

Sobre cuentas y permisos: `creapp-build` prepara el release, `creapp` ejecuta la aplicación y solo escribe en `backend/staticfiles`, y `www-data` lee código y estáticos. `/etc/creapp/creapp.env` es `root:creapp` con modo

0640, de modo que el grupo del servidor web no puede leer los secretos. Estas relaciones se verificaron en un contenedor descartable, no en una instalación real.

`systemd-run` se usa aquí por conveniencia para pasar `EnvironmentFile` sin leer secretos en una shell. Si la política del sistema no permite unidades transitorias, defina una unidad `oneshot` equivalente. Los avisos `Underfull` de un archivo no habilitado no son fallos del procedimiento.

3. Seguridad y entorno

HSTS. Está desactivado por defecto. Un `check --deploy` aislado con `SECURE_HSTS_SECONDS=0` informó W004; con 3600 y las subopciones en falso, informó W005 y W021. Son advertencias de configuración: habilite HSTS solo después de confirmar HTTPS, el dominio y todos los subdominios, y hágalo de forma gradual. No active `includeSubDomains` ni `preload` únicamente para silenciar la salida.

TLS hacia PostgreSQL. `settings.py` usa parámetros separados y no interpreta `POSTGRES_SSLMODE`. Para una base remota configure `PGSSLMODE` y `PGSSLROOTCERT` en el entorno del proceso, que `libpq` sí reconoce, y verifique la validación del certificado.

Proxy de acceso. `ProxyAccessMiddleware` solo se activa si se define `PROXY_ACCESS_SECRET`; entonces exige la cabecera `X-Proxy-Access-Secret` salvo en las rutas exentas, que por defecto incluyen `/healthz`, el panel y los estáticos. El cliente usa cookie en lugar de cabecera. En el repositorio existe además `frontend/middleware.js`, que inyecta esa cabecera con `@vercel/functions`; es una función de edge y no implica que un runtime de Vercel esté configurado. Solo actívelo con un proxy de confianza diseñado para ello.

Dependencias frontend. `npm ci` informó 16 avisos: 1 bajo, 4 medios y 11 altos. No se aplicó `npm audit fix` automáticamente. Revíselos por separado: una actualización automática de dependencias cambia el comportamiento de compilación y debe probarse.

4. Migraciones con historial

`0011_seed_ip_tipo_actividad` crea los nombres de tipo IP que faltan. En versiones anteriores de ese archivo, un nombre existente podía ver su dedicación reescrita a IP. La corrección publicada en la rama de producto evita tanto el seed como el importador, y el importador ahora falla con un error claro ante una dedicación distinta. Ninguna de las dos cosas repara una base que ya ejecutó la versión anterior: el efecto histórico solo puede corregirse con una decisión explícita de Retención o reinicio, no con una nueva migración.

0012 agrega cuatro campos de texto a `Programa` y no crea relaciones de competencias. El nombre de la migración puede sugerir lo contrario.

El plan de liberación de Azure y el resguardo de datos anteriores a la migración 0009 se conservan en `../AZURE_RELEASE_PLAN.md` y `../LEGACY_MODALIDAD_MIGRATION.md`. Ambos registran observaciones de una auditoría previa, no un estado verificado en el momento de leerlos.

5. Deuda técnica conocida

- **ClaseCalendarioSerializer no valida el ámbito docente.** Acepta `programa` y `dia_clase` sin comprobar la asignación, tanto al crear como al actualizar. El `queryset` filtra la lectura, pero la escritura puede apuntar a otro programa. La acción `generar-rango` sí valida.
- **Solapamiento de asignaciones.** La detección de períodos coincidentes es una validación de Python que lee y luego escribe. Dos peticiones concurrentes pueden pasar el chequeo. Faltan una restricción de exclusión o un bloqueo transaccional, y pruebas de concurrencia.
- **Endpoints temporales de docente.** `temporal/espacio`, `temporal/carga-horaria` y `temporal/asignarme` crean y modifican catálogo, relación curricular y asignaciones reales. Son una prueba funcional y deben retirarse cuando exista la integración institucional. Exigen perfil `DOCENTE`, no solo rol activo, por lo que un usuario `staff` con perfil docente también puede usarlos.
- **Créditos en la vista.** La pantalla de espacios calcula con 27 horas por crédito cuando aún no recibió la configuración, mientras el backend usa el valor configurado y 25 por omisión. La vista puede mostrar un número distinto al que cálculo el servidor.

- **Catálogo de tipos.** La distinción IP/TA en `TipoActividad.tipo_dedicacion` condiciona serializers, listas, formularios y calendario. Cambiarla exige unificar esos puntos.
- **Credenciales de docentes importados.** `import_academic_xlsx` deja contraseñas inutilizables; hace falta un flujo administrativo para establecerlas.
- **Pruebas con nombre desactualizado.** `PasswordValidationGapTest` en `accounts/tests.py` describe una brecha que ya no existe: ambos serializers validan la contraseña. Corrija el nombre y los comentarios al tocar esa suite.
- **Inventario de pruebas del backend.** Además de `accounts`, `academics` y `planning`, existen `config/tests_csrf.py` y `config/tests_proxy_access.py`. El segundo prueba rutas históricas del panel.
- **Documentación de API desactualizada.** `docs/API.md` usa barra final, recursos inexistentes y filtros no implementados. El router es la referencia.
- **Integración SIU no implementada.** Los documentos que describen un contrato con SIU Guaraní son una solicitud; no existe cliente ni endpoints en el backend. El flujo comprobable es la importación de XLSX.

6. Contratos de serialización

No todos los atributos del modelo están expuestos. Antes de asumir que un campo existe en la API, revise la lista `fields` del serializer.

- Las claves foráneas se devuelven como identificadores enteros, no como objetos anidados.
- **Programa** acepta plan-espacio, año y textos al crear; al actualizar solo el año y los textos, de modo que no se mueve de plan-espacio.
- **Unidad** acepta programa, número y descripción; al actualizar solo número y descripción. Las competencias se asignan por la acción dedicada, que reemplaza el conjunto completo y conserva el orden recibido.
- **Actividad** exige unidades activas del mismo programa. Para interacción pedagógica se exige clase del mismo programa y que el día esté habilitado si hay reglas; no admite fechas de trabajo autónomo ni horas por encima de la franja del día. Para trabajo autónomo se prohíbe la clase de calendario y se valida el orden de las fechas. Al actualizar, `unidad_ids` es opcional y si se omite conserva la relación.
- **DiaClasePrograma** exige que la hora de inicio y la de fin estén ambas presentes o ambas vacías.
- **AsignacionDocente** rechaza usuarios inactivos o que no son docentes, fechas invertidas y períodos solapados para el mismo docente y espacio. El campo `activo` es un valor calculado por fecha, no una columna.
- **ActividadAjuste** exige horas extra por encima de cero cuando el tipo es extensión, y que la clase destino pertenezca al programa de la actividad.
- Las horas se almacenan como decimales. Las pantallas capturan minutos y convierten antes de enviar.

7. Detalle del cliente frontend

- El interceptor de `services/api.ts` adjunta `X-CSRFToken` desde el token guardado en una respuesta o desde la cookie legible, emite un evento global de cambio de datos tras cada mutación y redirige al login ante errores de autenticación, excepto durante la comprobación de sesión y en la propia pantalla de login. No todo 403 es sesión expirada: la detección comprueba el detalle del mensaje.
- `useApiAutoRefresh` vuelve a pedir datos con retardo tras ese evento, al enfocar la ventana y al volver a ser visible el documento. Las pantallas con React Query invalidan además sus claves; el evento no invalida la caché por sí solo.
- `AdminCrudPage` resuelve campos opcionales con `emptyAs` cuando el serializer espera `null`, y usa `valueType: 'number'` para claves foráneas numéricas. Su validación de obligatoriedad es superficial: el backend sigue siendo la autoridad.
- `useDocenteSelection` guarda el espacio, el plan y su nombre en el almacenamiento de sesión, y notifica el cambio con un evento local. Otras pestañas no comparten ese estado de forma automática.
- `src/utils/errors.ts` traduce errores de red y de campo a mensajes en español; conviene usarlo en lugar de mostrar el error crudo.
- Tailwind está instalado, pero la mayoría de los estilos viven en hojas de CSS propias con tokens. Respete la convención del archivo del componente antes de introducir variables nuevas.

8. Evidencia de las pruebas

El recorrido de aceptación se ejecutó en contenedores descartables, contra una copia temporal del código elegido. Pasaron 54 pruebas frontend, 78 pruebas Django, las migraciones desde base vacía, la comprobación de estáticos, una sesión real con CSRF, el panel de administración, y un ciclo de respaldo y restauración con coincidencia del recuento de la tabla usada en la prueba.

Límites: no cubre Azure, una VM con systemd, DNS o certificados públicos, PostgreSQL remoto, firewall institucional ni el comportamiento real de cookies en navegadores. La comparación del respaldo se limitó a una tabla, no a todos los datos. El tiempo de ejecución de la corrida no es una estimación de capacidad.

Dos detalles del propio harness que conviene conocer: el cliente de pruebas de Django usa HTTP local, por lo que solo ese proceso desactiva la redirección a HTTPS, mientras el servicio bajo prueba la mantiene; y un intento inicial falló con 400 por no incluir el host de prueba en `ALLOWED_HOSTS`, un defecto del fixture que no cambió configuración del producto.

El harness es reproducible y está en [validation/](#). Ejecuta pruebas, no despliega.

9. Diagramas

Las fuentes editables están en [diagrams/](#), en formato TikZ con sus PDF y SVG.

- Catálogo académico y planificación: `classes-academics`, `classes-planning`, `classes-calendar`, `classes-identity`. No incluyen `ConfiguracionCRE`, que se documenta en la tabla del manual de desarrollo.
- Topología del servidor propio: `deployment`.
- Rutas HTTP: `http-routing`.
- Flujos de despliegue: `deployment-workflows`.

10. Documentos relacionados

- [DEPLOYMENT_MANUAL.md](#): procedimiento de despliegue.
- [DEVELOPER_MANUAL.md](#): arquitectura y contratos internos.
- [../AZURE_RELEASE_PLAN.md](#): controles de liberación en Azure.
- [../LEGACY_MODALIDAD_MIGRATION.md](#): resguardo de datos anteriores a la migración 0009.